

NSI

Première

Édition de travail 2026-2027

01001110 01010011 01001001
01001110 01000101 01011000
01010101 01010011 00100001

Sommaire

1	Représentation des données : types et valeurs de base	1
2	Types construits : p-uplets, tableaux, dictionnaires	3
3	Traitement de données en tables	6
4	Langages et programmation	9
5	Algorithmique 1 : parcours et tris	12
6	Algorithmique 2 : dichotomie, algorithmes gloutons, k plus proches voisins	14
7	Interactions entre l'homme et la machine sur le Web	17
8	Architecture de von Neumann et systèmes d'exploitation	20
9	Réseaux : transmission de données	23
10	Projet et méthodes	25

1 Représentation des données : types et valeurs de base

Corrigé

1NSI-TB-EVAL-A-EX1

1. Divisions par 2 : $90 = 2 \times 45 + 0$, $45 = 2 \times 22 + 1$, $22 = 2 \times 11 + 0$, $11 = 2 \times 5 + 1$, $5 = 2 \times 2 + 1$, $2 = 2 \times 1 + 0$, $1 = 2 \times 0 + 1$. Restes de bas en haut : $90 = 1011010_2$. En base 16 : $90 = 16 \times 5 + 10$ (10 = A), soit $90 = 5A_{16}$.

2. $2^6 = 64 \leq 90 < 128 = 2^7$, donc 90 nécessite 7 bits.

3. $7 = 0000111_2$, soit sur 8 bits : 00000111. Inversion des bits : 11111000. Ajout de 1 : 11111001. Donc -7 s'écrit 11111001.

Corrigé

1NSI-TB-EVAL-A-EX2

1. Le test renvoie False. Chaque 0.1 est stocké de façon approchée en mémoire ; après trois additions, la valeur accumulée (0.30000000000000004) n'est pas identique bit à bit à la représentation flottante de 0.3.

2.

<i>a</i>	<i>b</i>	résultat
V	V	F
V	F	V
F	V	V
F	F	F

Cette expression reproduit le **ou exclusif** (xor) : elle est vraie lorsque *a* et *b* ont des valeurs différentes.

3. Oui, les trois encodages donnent 4 octets. Le mot "info" ne contient que des lettres non accentuées, présentes à l'identique dans ASCII (7 bits), ISO-8859-1 (8 bits, compatible ASCII sur ses 128 premiers caractères) et UTF-8 (qui code les 128 premiers caractères Unicode sur un seul octet, identique à ASCII).

Évaluation A — Types et valeurs de base

Durée : 45 minutes. Calculatrice et brouillon autorisés, pas d'ordinateur.

Barème indicatif : Ex. 1 : 10 pts (bases, complément à 2) ; Ex. 2 : 10 pts (flottants, booléens, encodage)

Exercice 1 — Bases et complément à 2

(10 points)

- (4 pts) Convertir 90 (écriture décimale) en base 2 puis en base 16, en détaillant les calculs.
- (3 pts) Combien de bits sont nécessaires pour écrire 90 en binaire ?
- (3 pts) Écrire -7 en complément à 2 sur 8 bits, en détaillant la méthode (écriture de 7, inversion des bits, ajout de 1).

Exercice 2 — Flottants, booléens, encodage

(10 points)

- (3 pts) Que renvoie le test $0.1 + 0.1 + 0.1 == 0.3$ en Python ? Expliquer pourquoi.
- (4 pts) Dresser la table de vérité de l'expression (*a* and not *b*) or (not *a* and *b*). Quel opérateur booléen usuel cette expression reproduit-elle ?

3. (3 pts) Le mot "info" (lettres non accentuées) occupe-t-il le même nombre d'octets une fois encodé en ASCII, en ISO-8859-1 et en UTF-8 ? Justifier.

1NSI-TB-EVAL-B-EX1

Corrigé

1. Divisions par 2 : $150 = 2 \times 75 + 0$, $75 = 2 \times 37 + 1$, $37 = 2 \times 18 + 1$, $18 = 2 \times 9 + 0$, $9 = 2 \times 4 + 1$, $4 = 2 \times 2 + 0$, $2 = 2 \times 1 + 0$, $1 = 2 \times 0 + 1$. Restes de bas en haut : $150 = 10010110_2$. En base 16 : $150 = 16 \times 9 + 6$, soit $150 = 96_{16}$.

2. $2^7 = 128 \leq 150 < 256 = 2^8$, donc 150 nécessite 8 bits.

3. $20 = 00010100_2$ sur 8 bits. Inversion des bits : 11101011. Ajout de 1 : 11101100. Donc -20 s'écrit 11101100.

1NSI-TB-EVAL-B-EX2

Corrigé

1. Le test renvoie False : 0.2 n'a pas d'écriture binaire finie, et l'accumulation de trois additions produit une valeur (0.6000000000000001) légèrement différente de la représentation flottante de 0.6. Une réécriture correcte : $\text{abs}((0.2+0.2+0.2) - 0.6) < 1e-9$.

2.

a	b	c	résultat
V	V	V	V
V	V	F	F
V	F	V	V
V	F	F	F
F	V	V	V
F	V	F	F
F	F	V	F
F	F	F	F

L'expression est vraie pour 3 combinaisons sur 8.

3. Non. En UTF-8, "vélo" occupe 5 octets (le « é » occupe deux octets en UTF-8, les trois autres lettres un octet chacune). En ISO-8859-1, il occupe 4 octets (chaque caractère, y compris « é », est codé sur un seul octet dans cet encodage).

Évaluation B — Types et valeurs de base

Durée : 45 minutes. Calculatrice et brouillon autorisés, pas d'ordinateur.

Barème indicatif : Ex. 1 : 10 pts (bases, complément à 2); Ex. 2 : 10 pts (flottants, booléens, encodage)

Exercice 1 — Bases et complément à 2

(10 points)

- (4 pts) Convertir 150 (écriture décimale) en base 2 puis en base 16, en détaillant les calculs.
- (3 pts) Combien de bits sont nécessaires pour écrire 150 en binaire ?
- (3 pts) Écrire -20 en complément à 2 sur 8 bits, en détaillant la méthode.

Exercice 2 — Flottants, booléens, encodage

(10 points)

- (3 pts) Que renvoie le test $0.2 + 0.2 + 0.2 == 0.6$ en Python ? Expliquer pourquoi, et proposer une réécriture correcte de ce test.
- (4 pts) Dresser la table de vérité de l'expression $(a \text{ or } b) \text{ and } c$. Pour combien de combinaisons de (a, b, c) cette expression est-elle vraie ?
- (3 pts) Le mot "vélo" occupe-t-il le même nombre d'octets une fois encodé en ISO-8859-1 et en UTF-8 ? Justifier précisément la valeur trouvée pour chaque encodage.

2 Types construits : p-uplets, tableaux, dictionnaires

Évaluation A — Types construits

Durée : 55 minutes. Calculatrice interdite. Documents interdits.

Barème indicatif : Ex. 1 : 4 pts (analyser) ; Ex. 2 : 6 pts (programmer) ; Ex. 3 : 5 pts (analyser, programmer) ; Ex. 4 : 5 pts (programmer)

Exercice 1 — Lecture de programme

(4 points)

On considère le programme suivant :

```
1 joueurs = [  
2     {"pseudo": "Ace", "score": 150},  
3     {"pseudo": "Bob", "score": 200},  
4     {"pseudo": "Cat", "score": 180},  
5 ]  
6 meilleur = joueurs[0]  
7 for j in joueurs:  
8     if j["score"] > meilleur["score"]:  
9         meilleur = j  
10 print(meilleur["pseudo"])  
11 print(meilleur["score"])
```

1. Donner, sans exécuter le programme, les deux valeurs affichées.
2. Expliquer en une phrase le rôle de ce programme.
3. Si on ajoute `meilleur["score"] = 0` après la boucle, quelle est la valeur de `joueurs[1]["score"]` ? Justifier.

Exercice 2 — Programmation avec des tableaux

(6 points)

Un tournoi de e-sport enregistre les scores de chaque manche dans un tableau :

```
1 scores = [150, 200, 180, 210, 170]
```

1. Écrire une fonction `au_dessus(scores, seuil)` qui renvoie le tableau des scores strictement supérieurs au seuil.
2. Écrire une fonction `moyenne(scores)` qui renvoie la moyenne des scores. La fonction doit lever `ValueError` si le tableau est vide.
3. Écrire une fonction `classement(scores)` qui renvoie le tableau des tuples (rang, score) triés par score décroissant. Exemple : `classement([150, 200, 180])` renvoie `[(1, 200), (2, 180), (3, 150)]`.

Exercice 3 — Grille et pixels

(5 points)

On représente une image en niveaux de gris par une grille (tableau de tableaux) :

```
1 image = [  
2     [100, 200, 100],  
3     [200, 50, 200],
```

```
4 [100, 200, 100],
5 ]
```

1. Combien de lignes et de colonnes contient cette image ?
2. Quelle est la valeur du pixel central ? Donner l'expression Python.
3. Écrire une fonction `nb_sombres(image, seuil)` qui compte le nombre de pixels dont la valeur est inférieure ou égale au seuil.

Exercice 4 — Dictionnaire et mutabilité

(5 points)

On gère un carnet de contacts :

```
1 contacts = {"Ali": "55123456", "Sara": "55234567"}
```

1. Écrire une fonction `chercher(contacts, nom)` qui renvoie le numéro du contact ou "Inconnu" si le nom n'est pas dans le carnet.
2. Écrire une fonction `ajouter(contacts, nom, numero)` qui ajoute un contact au carnet. Préciser : cette fonction modifie-t-elle le dictionnaire passé en argument ?
3. On exécute :

```
1 carnet1 = {"Ali": "55123456"}
2 carnet2 = carnet1
3 carnet2["Sara"] = "55234567"
```

Quelle est la valeur de `carnet1` après ces trois lignes ? Justifier.

Évaluation B — Types construits

Durée : 55 minutes. Calculatrice interdite. Documents interdits.

Barème indicatif : Ex. 1 : 5 pts (analyser) ; Ex. 2 : 5 pts (programmer) ; Ex. 3 : 5 pts (programmer, analyser) ; Ex. 4 : 5 pts (programmer)

Exercice 1 — Analyse de programme

(5 points)

On donne le programme suivant :

```
1 inventaire = {"pommes": 12, "bananes": 5, "oranges": 8}
2 panier = []
3 for fruit, quantite in inventaire.items():
4     if quantite > 6:
5         panier.append((fruit, quantite))
6 inventaire["pommes"] = 0
7 print(panier)
8 print(inventaire["pommes"])
```

1. Donner la valeur de `panier` après la boucle.
2. Quel est le type de chaque élément de `panier` ?
3. Après `inventaire["pommes"] = 0`, la valeur 12 dans `panier` est-elle modifiée ? Justifier.
4. Écrire en une ligne une compréhension de liste produisant le même résultat que la boucle.

Exercice 2 — Fonctions sur les tableaux

(5 points)

1. Écrire une fonction `separer(tab, seuil)` qui prend un tableau de nombres et renvoie un tuple de deux tableaux : les éléments $\leq$ au seuil et ceux $>$ au seuil.

```
»> separer([3, 8, 1, 5, 9, 2], 4)
([3, 1, 2], [8, 5, 9])
```

2. Écrire une fonction `sans_doublons(tab)` qui renvoie un nouveau tableau contenant les éléments de `tab` sans doublons, dans l'ordre de première apparition.

```
»> sans_doublons([3, 1, 3, 2, 1, 4])
[3, 1, 2, 4]
```

Exercice 3 — Grille de jeu

(5 points)

On représente une grille de morpion par un tableau de tableaux :

```
1 grille = [
2     ["X", "O", " "],
3     [" ", "X", " "],
4     ["O", " ", "X"],
5 ]
```

1. Écrire une fonction `compter(grille, symbole)` qui renvoie le nombre d'occurrences de symbole dans la grille.
2. Écrire une fonction `diagonale(grille)` qui renvoie le tableau des éléments de la diagonale principale.
3. La diagonale de l'exemple contient-elle trois symboles identiques ? Écrire une expression Python pour le vérifier.

Exercice 4 — Dictionnaires et mutabilité

(5 points)

On gère un carnet de notes par matière :

```
1 carnet = {"maths": [15, 12], "nsi": [18, 16, 14]}
```

1. Écrire une fonction `ajouter_note(carnet, matiere, note)` qui ajoute une note à la matière donnée. Si la matière n'existe pas, elle est créée avec cette seule note.
2. Écrire une fonction `moyennes(carnet)` qui renvoie un nouveau dictionnaire avec la moyenne par matière.
3. On exécute :

```
1 c1 = {"nsi": [18]}
2 c2 = c1
3 c2["nsi"].append(15)
```

Quelle est la valeur de `c1["nsi"]` ? Expliquer.

3 Traitement de données en tables

1NSI-TAB-EVAL-A-EX1

Corrigé

1. Avant conversion, `films[0]["annee"]` serait une chaîne de caractères (str), par exemple "2014".

```
1 for ligne in films:
2     ligne["annee"] = int(ligne["annee"])
3     ligne["note"] = float(ligne["note"])
```

- 2.

```
1 resultat = [f for f in films if f["genre"] == "SF" and f["note"] >= 8.5]
```

3. 2 films remplissent ce critère : **Interstellar** (note 8,6) et **Inception** (note 8,8).

1NSI-TAB-EVAL-A-EX2

Corrigé

1. `sorted(films, key=lambda ligne: ligne["annee"])`. Le premier film après tri est **Amelie** (2001, la plus ancienne année).
2. En fusionnant films et realisateurs sur titre, on obtient 3 lignes (Interstellar, Inception, Amelie).
3. **Dune** n'apparaît pas dans le résultat : son titre n'est pas présent dans la table realisateurs, il est donc hors du domaine de valeurs commun aux deux tables sur la colonne titre.

Évaluation A — Traitement de données en tables

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (import, recherche); Ex. 2 : 10 pts (tri, fusion)

Exercice 1 — Une table de films

(10 points)

```
1 films = [
2     {"titre": "Interstellar", "annee": 2014, "genre": "SF", "note": 8.6},
3     {"titre": "Inception", "annee": 2010, "genre": "SF", "note": 8.8},
4     {"titre": "Amelie", "annee": 2001, "genre": "Comedie", "note": 8.3},
5     {"titre": "Dune", "annee": 2021, "genre": "SF", "note": 8.0},
6 ]
```

1. (3 pts) Si cette table provenait d'un fichier CSV importé avec `csv.DictReader`, quel serait le type de `films[0]["annee"]` avant conversion ? Écrire le code de conversion nécessaire pour les colonnes annee (en int) et note (en float).
2. (4 pts) Écrire une compréhension de liste renvoyant les films de genre "SF" ayant une note supérieure ou égale à 8,5.
3. (3 pts) Combien de films remplissent ce critère ? Les nommer.

Exercice 2 — Tri et fusion

(10 points)

1. (3 pts) Écrire l'instruction qui trie la table films par année croissante. Donner le titre du premier film après ce tri.

2. (4 pts) On dispose de la table realisateurs :

```
1 realisateurs = [  
2     {"titre": "Interstellar", "realisateur": "Nolan"},  
3     {"titre": "Inception", "realisateur": "Nolan"},  
4     {"titre": "Amelie", "realisateur": "Jeunet"},  
5 ]
```

En utilisant la fonction fusionner(table1, table2, cle) du cours, fusionner films et realisateurs sur titre. Combien de lignes obtient-on ?

3. (3 pts) Quel film de la table films n'apparaît pas dans le résultat de la fusion ? Pourquoi ?

Corrigé

1NSI-TAB-EVAL-B-EX1

1. Avant conversion, jeux[0]["note"] serait une chaîne de caractères (str), par exemple "9.7".

```
1 for ligne in jeux:  
2     ligne["annee"] = int(ligne["annee"])  
3     ligne["note"] = float(ligne["note"])
```

2.

```
1 resultat = [j for j in jeux if j["genre"] == "Puzzle" and j["note"] >= 9.3]
```

3. 1 seul jeu remplit ce critère : **Portal 2** (note 9,5). Tetris, bien que du genre Puzzle, a une note de 9,0, inférieure au seuil de 9,3.

Corrigé

1NSI-TAB-EVAL-B-EX2

1. sorted(jeux, key=lambda ligne: ligne["annee"]). Le premier jeu après tri est **Tetris** (1984, la plus ancienne année).

2. En fusionnant jeux et studios sur titre, on obtient 3 lignes (Zelda, Portal 2, Celeste).

3. **Tetris** n'apparaît pas dans le résultat : son titre n'est pas présent dans la table studios, il est donc hors du domaine de valeurs commun aux deux tables sur la colonne titre.

Évaluation B — Traitement de données en tables

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (import, recherche) ; Ex. 2 : 10 pts (tri, fusion)

Exercice 1 — Une table de jeux vidéo

(10 points)

```
1 jeux = [  
2     {"titre": "Zelda", "annee": 2017, "genre": "Aventure", "note": 9.7},  
3     {"titre": "Portal 2", "annee": 2011, "genre": "Puzzle", "note": 9.5},  
4     {"titre": "Celeste", "annee": 2018, "genre": "Plateforme", "note": 9.2},  
5     {"titre": "Tetris", "annee": 1984, "genre": "Puzzle", "note": 9.0},  
6 ]
```

1. (3 pts) Si cette table provenait d'un fichier CSV importé avec csv.DictReader, quel serait le type de jeux[0]["note"] avant conversion ? Écrire le code de conversion nécessaire pour les colonnes annee (en int) et note (en float).
2. (4 pts) Écrire une compréhension de liste renvoyant les jeux de genre "Puzzle" ayant une note supérieure ou égale à 9,3.
3. (3 pts) Combien de jeux remplissent ce critère ? Les nommer.

Exercice 2 — Tri et fusion

(10 points)

1. (3 pts) Écrire l'instruction qui trie la table jeux par année croissante. Donner le titre du premier jeu après ce tri.
2. (4 pts) On dispose de la table studios :

```
1 studios = [
2     {"titre": "Zelda", "studio": "Nintendo"},
3     {"titre": "Portal 2", "studio": "Valve"},
4     {"titre": "Celeste", "studio": "Matt Makes Games"},
5 ]
```

En utilisant la fonction `fusionner(table1, table2, cle)` du cours, fusionner jeux et studios sur titre. Combien de lignes obtient-on ?

3. (3 pts) Quel jeu de la table jeux n'apparaît pas dans le résultat de la fusion ? Pourquoi ?

4 Langages et programmation

Corrigé

1NSI-LANG-EVAL-A-EX1

1. Précondition : valeurs et poids ont la même longueur, et la somme des poids est strictement positive (pour ne pas diviser par zéro).

2.

```
1 def moyenne_ponderee(valeurs, poids):
2     assert len(valeurs) == len(poids)
3     assert sum(poids) > 0
4     total = 0
5     for i in range(len(valeurs)):
6         total = total + valeurs[i] * poids[i]
7     return total / sum(poids)
```

$$3. \frac{12 \times 1 + 15 \times 2 + 8 \times 1}{1 + 2 + 1} = \frac{12 + 30 + 8}{4} = \frac{50}{4} = 12,5.$$

Corrigé

1NSI-LANG-EVAL-A-EX2

1. somme_positifs_bugue([3, -2, 5]) renvoie 3. Le résultat attendu est 8 (3 + 5, les deux éléments strictement positifs).

2. L'erreur est dans range(len(liste) - 1) : cette boucle parcourt les indices de 0 à len(liste) - 2, donc **exclut le dernier élément** de la liste (indice len(liste) - 1). Version corrigée :

```
1 def somme_positifs(liste):
2     total = 0
3     for i in range(len(liste)):
4         if liste[i] > 0:
5             total += liste[i]
6     return total
```

3. import math puis math.gcd(84, 30), qui renvoie 6.

Évaluation A — Langages et programmation

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (constructions, spécification) ; Ex. 2 : 10 pts (mise au point, bibliothèque)

Exercice 1 — Moyenne pondérée

(10 points)

- (2 pts) Proposer une précondition pour une fonction moyenne_ponderee(valeurs, poids) calculant une moyenne pondérée (on pourra penser à la longueur des deux listes, et à la somme des poids).
- (5 pts) Écrire cette fonction, avec les assert correspondants, en utilisant une boucle for bornée.
- (3 pts) Calculer moyenne_ponderee([12, 15, 8], [1, 2, 1]) à la main, puis vérifier avec le code.

Exercice 2 — Mise au point et bibliothèque

(10 points)

Un élève écrit la fonction suivante, censée renvoyer la somme des éléments strictement positifs d'une liste :

```
1 def somme_positifs_bugue(liste):
2     total = 0
3     for i in range(len(liste) - 1):
4         if liste[i] > 0:
5             total += liste[i]
6     return total
```

1. (3 pts) Que renvoie `somme_positifs_bugue([3, -2, 5])` ? Quel est le résultat attendu ?
2. (4 pts) Identifier précisément l'erreur dans le code, et proposer une version corrigée.
3. (3 pts) En utilisant le module `math`, calculer le PGCD de 84 et 30 (indiquer le nom de la fonction utilisée).

1NSI-LANG-EVAL-B-EX1

Corrigé

1. Précondition : valeurs et poids ont la même longueur, et la somme des poids est strictement positive.

2.

```
1 def moyenne_ponderee(valeurs, poids):
2     assert len(valeurs) == len(poids)
3     assert sum(poids) > 0
4     total = 0
5     for i in range(len(valeurs)):
6         total = total + valeurs[i] * poids[i]
7     return total / sum(poids)
```

$$3. \frac{8 \times 1 + 12 \times 1 + 16 \times 2}{1 + 1 + 2} = \frac{8 + 12 + 32}{4} = \frac{52}{4} = 13.$$

1NSI-LANG-EVAL-B-EX2

Corrigé

1. `produit_negatifs_bugue([-2, -3, -4])` renvoie 0. Le résultat attendu est $-24 ((-2) \times (-3) \times (-4))$.

2. L'accumulateur `prod` est initialisé à 0 : dès la première multiplication (`prod *= x`), $0 \times x = 0$ quel que soit x , donc `prod` reste 0 pour toujours. Il faut initialiser un accumulateur de **produit** à 1 (l'élément neutre de la multiplication), pas à 0 :

```
1 def produit_negatifs(liste):
2     prod = 1
3     for x in liste:
4         if x < 0:
5             prod *= x
6     return prod
```

3. `import math` puis `math.gcd(60, 48)`, qui renvoie 12.

Évaluation B — Langages et programmation

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (constructions, spécification) ; Ex. 2 : 10 pts (mise au point, bibliothèque)

Exercice 1 — Moyenne pondérée

(10 points)

1. (2 pts) Proposer une précondition pour une fonction `moyenne_ponderee(valeurs, poids)` calculant une moyenne pondérée.
2. (5 pts) Écrire cette fonction, avec les assert correspondants, en utilisant une boucle `for` bornée.
3. (3 pts) Calculer `moyenne_ponderee([8, 12, 16], [1, 1, 2])` à la main, puis vérifier avec le code.

Exercice 2 — Mise au point et bibliothèque

(10 points)

Un élève écrit la fonction suivante, censée renvoyer le produit des éléments strictement négatifs d'une liste :

```
1 def produit_negatifs_bugue(liste):  
2     prod = 0  
3     for x in liste:  
4         if x < 0:  
5             prod *= x  
6     return prod
```

1. (3 pts) Que renvoie `produit_negatifs_bugue([-2, -3, -4])` ? Quel est le résultat attendu ?
2. (4 pts) Identifier précisément l'erreur dans le code, et proposer une version corrigée.
3. (3 pts) En utilisant le module `math`, calculer le PGCD de 60 et 48 (indiquer le nom de la fonction utilisée).

5 Algorithmique 1 : parcours et tris

1NSI-AGT-EVAL-A-EX1

Corrigé

1. La valeur 30 se trouve à l'indice 3.
2. Le maximum est 30.
3. $\frac{22 + 15 + 8 + 30 + 11}{5} = \frac{86}{5} = 17,2$.

1NSI-AGT-EVAL-A-EX2

Corrigé

1. Tableau initial : [9, 2, 6, 4].
 - ▶ Après $i = 1$ (insertion de 2) : [2, 9, 6, 4]
 - ▶ Après $i = 2$ (insertion de 6) : [2, 6, 9, 4]
 - ▶ Après $i = 3$ (insertion de 4) : [2, 4, 6, 9]
2. Après $i = 1$: [2, 9] trié. Après $i = 2$: [2, 6, 9] trié. Après $i = 3$: [2, 4, 6, 9] trié. L'invariant est vérifié à chaque étape.
3. Le coût dans le pire cas est **quadratique**, $O(n^2)$.

Évaluation A — Parcours et tris

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (recherche, extremum, moyenne); Ex. 2 : 10 pts (tri par insertion, invariant)

Exercice 1 — Parcours séquentiel

(10 points)

On donne le tableau [22, 15, 8, 30, 11].

1. (3 pts) À quel indice se trouve la valeur 30 ? Utiliser `recherche_occurrence` du cours.
2. (3 pts) Calculer le maximum de ce tableau avec `maximum`.
3. (4 pts) Calculer la moyenne de ce tableau avec `moyenne`. Détailler le calcul à la main.

Exercice 2 — Tri par insertion

(10 points)

On applique le tri par insertion au tableau [9, 2, 6, 4].

1. (5 pts) Dérouler l'algorithme : donner le contenu du tableau après chaque itération de la boucle externe.
2. (3 pts) Vérifier, à chaque étape, l'invariant de boucle du cours (le préfixe traité est trié).
3. (2 pts) Quel est le coût de cet algorithme dans le pire cas, en fonction de la taille n du tableau ?

1NSI-AGT-EVAL-B-EX1

Corrigé

1. La valeur 6 se trouve à l'indice 2.
2. Le maximum est 40.
3. $\frac{17 + 40 + 6 + 25 + 12}{5} = \frac{100}{5} = 20$.

1. Tableau initial : [7, 1, 4, 9].

- ▶ $i = 0$: minimum de [7,1,4,9] à l'indice 1 (valeur 1). Après échange : [1, 7, 4, 9].
- ▶ $i = 1$: minimum de [7,4,9] à l'indice 2 (valeur 4). Après échange : [1, 4, 7, 9].
- ▶ $i = 2$: minimum de [7,9] à l'indice 2 (valeur 7, déjà en place). Après échange (avec elle-même) : [1, 4, 7, 9].

2. Après $i = 0$: [1] trié, $1 \leq 4$ (min du reste). Après $i = 1$: [1,4] trié, $4 \leq 7$. Après $i = 2$: [1,4,7] trié, $7 \leq 9$. L'invariant est vérifié à chaque étape.

3. Le coût dans le pire cas est **quadratique**, $O(n^2)$ (et c'est aussi le cas dans le **meilleur** cas pour ce tri, contrairement au tri par insertion).

Évaluation B — Parcours et tris

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (recherche, extremum, moyenne); Ex. 2 : 10 pts (tri par sélection, invariant)

Exercice 1 — Parcours séquentiel

(10 points)

On donne le tableau [17, 40, 6, 25, 12].

1. (3 pts) À quel indice se trouve la valeur 6 ? Utiliser recherche_occurrence du cours.
2. (3 pts) Calculer le maximum de ce tableau avec maximum.
3. (4 pts) Calculer la moyenne de ce tableau avec moyenne. Détailler le calcul à la main.

Exercice 2 — Tri par sélection

(10 points)

On applique le tri par sélection au tableau [7, 1, 4, 9].

1. (5 pts) Dérouler l'algorithme : pour chaque itération de la boucle externe, donner l'indice du minimum trouvé et le contenu du tableau après l'échange.
2. (3 pts) Vérifier, à chaque étape, l'invariant de boucle du cours (préfixe trié et inférieur ou égal au reste).
3. (2 pts) Quel est le coût de cet algorithme dans le pire cas, en fonction de la taille n du tableau ?

6 Algorithmique 2 : dichotomie, algorithmes gloutons, k plus proches voisins

1NSI-ADGK-EVAL-A-EX1

Corrigé

1. Itération 1 : $\text{inf}=0$, $\text{sup}=7$, $\text{milieu}=3$ ($t[3]=21 < 27$). Itération 2 : $\text{inf}=4$, $\text{sup}=7$, $\text{milieu}=5$ ($t[5]=33 > 27$). Itération 3 : $\text{inf}=4$, $\text{sup}=4$, $\text{milieu}=4$ ($t[4]=27$, trouvé, indice 4).
2. V vaut successivement $7 - 0 = 7$, $7 - 4 = 3$, $4 - 4 = 0$: la suite 7, 3, 0 est strictement décroissante.
3. V est un entier positif ou nul qui décroît strictement : une telle suite est nécessairement finie (elle ne peut pas décroître indéfiniment en restant positive), donc la boucle termine forcément après un nombre fini d'itérations.

1NSI-ADGK-EVAL-A-EX2

Corrigé

1. Pour 47 : 20 (reste 27), 20 (reste 7), 5 (reste 2), 2 (reste 0). Rendu : [20, 20, 5, 2].
2. `k_plus_proches_voisins(animaux, (5.5, 31), 3)` renvoie "Chat" : cet animal est proche en masse et en taille du groupe des chats.
3. Non, la méthode gloutonne n'est pas optimale pour **tout** système de pièces (seulement pour des systèmes ayant certaines propriétés, comme le système en euros) : avec {4, 3, 1} par exemple, rendre 6 de façon gloutonne utilise 3 pièces ($4 + 1 + 1$) alors que $3 + 3$ n'en utilise que 2.

Évaluation A — Dichotomie, gloutons, k-NN

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (dichotomie, variant) ; Ex. 2 : 10 pts (glouton, k-NN)

Exercice 1 — Recherche dichotomique

(10 points)

On applique `recherche_dichotomique` du cours au tableau trié [3, 9, 14, 21, 27, 33, 40, 48] pour chercher la valeur 27.

1. (5 pts) Dérouler l'algorithme : borne_inf, borne_sup, milieu à chaque itération, jusqu'à trouver la valeur.
2. (3 pts) Calculer le variant $V = \text{borne_sup} - \text{borne_inf}$ à chaque itération et vérifier qu'il décroît strictement.
3. (2 pts) Pourquoi cette preuve de décroissance stricte suffit-elle à garantir la terminaison de l'algorithme ?

Exercice 2 — Glouton et k-NN

(10 points)

1. (4 pts) Utiliser `rendu_glouton` du cours pour rendre 47 euros avec le système {20, 10, 5, 2, 1}. Détailler le choix de chaque pièce.
2. (3 pts) On dispose de la base d'animaux (masse en kg, taille en cm, espèce) :


```

1 animaux = [
2     (5, 30, "Chat"), (6, 32, "Chat"), (4, 28, "Chat"),
3     (30, 80, "Chien"), (32, 85, "Chien"), (28, 78, "Chien"),
4 ]

```

Prédire l'espèce d'un animal de masse 5,5 kg et de taille 31 cm, avec $k = 3$.

3. (3 pts) La méthode gloutonne de rendu de monnaie donne-t-elle toujours le nombre minimal de pièces, quel que soit le système de pièces ? Justifier brièvement.

Corrigé

1NSI-ADGK-EVAL-B-EX1

1. Itération 1 : $\text{inf}=0$, $\text{sup}=7$, $\text{milieu}=3$ ($t[3]=17 < 24$). Itération 2 : $\text{inf}=4$, $\text{sup}=7$, $\text{milieu}=5$ ($t[5]=31 > 24$). Itération 3 : $\text{inf}=4$, $\text{sup}=4$, $\text{milieu}=4$ ($t[4]=24$, trouvé, indice 4).

2. V vaut successivement $7 - 0 = 7$, $7 - 4 = 3$, $4 - 4 = 0$: la suite 7, 3, 0 est strictement décroissante.

3. V est un entier positif ou nul qui décroît strictement à chaque itération : une telle suite ne peut être infinie, donc la boucle se termine forcément.

Corrigé

1NSI-ADGK-EVAL-B-EX2

1. Pour 38 : 20 (reste 18), 10 (reste 8), 5 (reste 3), 2 (reste 1), 1 (reste 0). Rendu : [20, 10, 5, 2, 1].

2. `k_plus_proches_voisins(fleurs, (5.1, 1.5), 3)` renvoie "Iris" : cette fleur est proche en longueur et largeur de pétale du groupe des iris.

3. Avec $k = 1$, la prédiction ne repose que sur le **seul** voisin le plus proche : si ce point unique est mal classé dans la base (une erreur de saisie, un cas atypique), la prédiction sera directement faussée, sans qu'aucun autre voisin ne vienne « corriger » cette erreur par un vote majoritaire.

Évaluation B — Dichotomie, gloutons, k-NN

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (dichotomie, variant); Ex. 2 : 10 pts (glouton, k-NN)

Exercice 1 — Recherche dichotomique

(10 points)

On applique `recherche_dichotomique` du cours au tableau trié [2, 6, 11, 17, 24, 31, 39, 50] pour chercher la valeur 24.

- (5 pts) Dérouler l'algorithme : `borne_inf`, `borne_sup`, `milieu` à chaque itération, jusqu'à trouver la valeur.
- (3 pts) Calculer le variant $V = \text{borne_sup} - \text{borne_inf}$ à chaque itération et vérifier qu'il décroît strictement.
- (2 pts) Pourquoi cette preuve de décroissance stricte suffit-elle à garantir la terminaison de l'algorithme ?

Exercice 2 — Glouton et k-NN

(10 points)

- (4 pts) Utiliser `rendu_glouton` du cours pour rendre 38 euros avec le système {20, 10, 5, 2, 1}. Détailler le choix de chaque pièce.
- (3 pts) On dispose de la base de fleurs (longueur de pétale en cm, largeur de pétale en cm, espèce) :

```

1 fleurs = [
2     (5, 1.5, "Iris"), (5.2, 1.6, "Iris"), (4.8, 1.4, "Iris"),
3     (6.5, 2.2, "Rose"), (6.8, 2.3, "Rose"), (6.3, 2.1, "Rose"),
4 ]

```

Prédire l'espèce d'une fleur de longueur 5,1 cm et de largeur 1,5 cm, avec $k = 3$.

3. (3 pts) Un k trop petit (par exemple $k = 1$) présente quel risque, en présence de données bruitées (un point mal classé dans la base) ?

7 Interactions entre l'homme et la machine sur le Web

Corrigé

1NSI-WEB-EVAL-A-EX1

1. Non. Le HTML décrit uniquement l'apparence des composants ; sans code JavaScript associant une fonction à l'événement clic du bouton, rien ne se produit automatiquement.

2.

```
1 panier = {"articles": 0}
2
3 def ajouter_article():
4     panier["articles"] += 1
5     return panier["articles"]
6
7 ajouter_gestionnaire("bouton_ajouter", ajouter_article)
```

3. La séquence est [1, 2, 3].

Corrigé

1NSI-WEB-EVAL-A-EX2

1.

```
1 from urllib.parse import urlencode
2
3 params = {"q": "ordinateur", "trie": "popularite"}
4 url = "https://boutique.fr/recherche?" + urlencode(params)
5 print(url)
6 # https://boutique.fr/recherche?q=ordinateur&trie=popularite
```

2. Oui, GET est adapté : les critères de recherche ne sont pas confidentiels, et l'URL résultante peut être partagée ou mise en favori pour retrouver la même recherche.

3. Il faut privilégier **POST** : un numéro de téléphone et une adresse postale sont des données à caractère personnel, qui ne doivent pas apparaître dans l'URL (historique du navigateur, journaux du serveur).

Évaluation A — Interactions homme-machine sur le Web

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (composants, gestionnaire d'événement) ; Ex. 2 : 10 pts (GET/POST)

Exercice 1 — Bouton « ajouter au panier »

(10 points)

On donne le code HTML suivant :

```
<button id="bouton_ajouter">Ajouter au panier</button>
<span id="compteur">0</span> article(s)
```

1. (2 pts) Ce code HTML seul suffit-il à incrémenter le compteur lors d'un clic ? Justifier.

2. (5 pts) En reprenant le modèle du cours (ajouter_gestionnaire, declencher_clic), écrire une fonction ajouter_article() qui incrémente panier["articles"] (initialement 0) et renvoie la nouvelle valeur, puis l'associer à "bouton_ajouter".
3. (3 pts) Simuler 3 clics successifs. Quelle est la séquence de valeurs renvoyées ?

Exercice 2 — Choisir GET ou POST

(10 points)

1. (4 pts) Un formulaire de recherche de produits transmet les paramètres {"q": "ordinateur", "trie": "popularite"}. Construire, avec urlencode, l'URL de la requête GET correspondante (base : "https://boutique.fr/recherche").
2. (3 pts) Cette recherche peut-elle raisonnablement utiliser GET ? Justifier en citant la nature des données.
3. (3 pts) Un autre formulaire du même site transmet un numéro de téléphone et une adresse postale pour une livraison. Quelle méthode privilégier, et pourquoi ?

1NSI-WEB-EVAL-B-EX1

Corrigé

1. Non. Le HTML décrit uniquement l'apparence des composants ; sans code JavaScript associant une fonction à l'événement clic, rien ne se produit automatiquement.

2.

```

1 favoris = {"articles": 3}
2
3 def retirer_favori():
4     favoris["articles"] -= 1
5     return favoris["articles"]
6
7 ajouter_gestionnaire("bouton_retirer", retirer_favori)

```

3. La séquence est [2, 1].

1NSI-WEB-EVAL-B-EX2

Corrigé

1.

```

1 from urllib.parse import urlencode
2
3 params = {"ville": "Sousse", "type": "appartement"}
4 url = "https://immo.fr/recherche?" + urlencode(params)
5 print(url)
6 # https://immo.fr/recherche?ville=Sousse&type=appartement

```

2. Oui, GET est adapté : les critères de recherche (ville, type de bien) ne sont pas confidentiels, et l'URL résultante peut être partagée ou mise en favori.

3. Il faut privilégier **POST** : le nom, l'email et le message d'un visiteur sont des données à caractère personnel qui ne doivent pas apparaître dans l'URL.

Évaluation B — Interactions homme-machine sur le Web

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (composants, gestionnaire d'événement) ; Ex. 2 : 10 pts (GET/POST)

Exercice 1 — Bouton « retirer des favoris »

(10 points)

On donne le code HTML suivant, sur une page listant 3 favoris :

```
<button id="bouton_retirer">Retirer des favoris</button>  
<span id="compteur">3</span> favori(s)
```

1. (2 pts) Ce code HTML seul suffit-il à décrémenter le compteur lors d'un clic ? Justifier.
2. (5 pts) En reprenant le modèle du cours, écrire une fonction `retirer_favori()` qui décrémente `favoris["articles"]` (initialisé à 3) et renvoie la nouvelle valeur, puis l'associer à "bouton_retirer".
3. (3 pts) Simuler 2 clics successifs. Quelle est la séquence de valeurs renvoyées ?

Exercice 2 — Choisir GET ou POST

(10 points)

1. (4 pts) Un formulaire de recherche immobilière transmet les paramètres `{"ville": "Sousse", "type": "appartement"}`. Construire, avec `urlencode`, l'URL de la requête GET correspondante (base : `"https://immo.fr/recherche"`).
2. (3 pts) Cette recherche peut-elle raisonnablement utiliser GET ? Justifier en citant la nature des données.
3. (3 pts) Un autre formulaire du même site transmet le nom, l'email et le message d'un visiteur souhaitant être recontacté. Quelle méthode privilégier, et pourquoi ?

8 Architecture de von Neumann et systèmes d'exploitation

1NSI-ARCHOS-EVAL-A-EX1

Corrigé

1. Processeur, mémoire, entrées, sorties. La **mémoire** stocke à la fois les instructions et les données.
- 2.

```
1 def executer(programme, memoire):
2     registres = {}
3     for instruction in programme:
4         op = instruction[0]
5         if op == "LOAD":
6             _, reg, adresse = instruction
7             registres[reg] = memoire[adresse]
8         elif op == "MUL":
9             _, reg_dest, reg_src = instruction
10            registres[reg_dest] = registres[reg_dest] * registres[reg_src]
11        elif op == "STORE":
12            _, reg, adresse = instruction
13            memoire[adresse] = registres[reg]
14        elif op == "HALT":
15            break
16    return memoire, registres
```

3. Après LOAD R0,0 : R0=6. Après LOAD R1,1 : R1=7. Après MUL R0,R1 : R0=42. Après STORE R0,2 : memoire=[6, 7, 42].

1NSI-ARCHOS-EVAL-A-EX2

Corrigé

1. Par exemple : gestion des processus (partage du temps processeur) et gestion des droits d'accès aux fichiers (autres réponses possibles : gestion de la mémoire, gestion des fichiers).
2. Le propriétaire **peut** exécuter (rwx). Le groupe **peut** exécuter (r-x). Les autres **ne peuvent pas** lire (---).
3. rwx=7, r-x=5, ---=0 : notation octale 750.

Évaluation A — Architecture et systèmes d'exploitation

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (von Neumann, instructions) ; Ex. 2 : 10 pts (OS, permissions)

Exercice 1 — Von Neumann et instructions machine

(10 points)

1. (3 pts) Nommer les quatre constituants principaux de l'architecture de von Neumann, et préciser lequel stocke à la fois instructions et données.
2. (4 pts) On ajoute au simulateur executer du cours l'instruction ("MUL", reg_dest, reg_src) (multiplication : $\text{reg_dest} \leftarrow \text{reg_dest} * \text{reg_src}$). Écrire le code de cette instruction.
3. (3 pts) Dérouler l'exécution de ce programme et donner l'état final de memoire :

```

1 memoire = [6, 7, 0]
2 programme = [
3     ("LOAD", "R0", 0),
4     ("LOAD", "R1", 1),
5     ("MUL", "R0", "R1"),
6     ("STORE", "R0", 2),
7     ("HALT", ),
8 ]

```

Exercice 2 — Système d'exploitation et permissions

(10 points)

- (3 pts) Citer deux fonctions assurées par un système d'exploitation.
- (4 pts) Un fichier a les permissions "rwxr-x---". En utilisant droit_accorde du cours, déterminer si le propriétaire peut l'exécuter, si le groupe peut l'exécuter, et si les autres peuvent le lire.
- (3 pts) Convertir ces permissions en notation octale avec permissions_vers_octal.

Corrigé

1NSI-ARCHOS-EVAL-B-EX1

1. Processeur, mémoire, entrées, sorties. Le **processeur** exécute les instructions.

2.

```

1 def executer(programme, memoire):
2     registres = {}
3     for instruction in programme:
4         op = instruction[0]
5         if op == "LOAD":
6             _, reg, adresse = instruction
7             registres[reg] = memoire[adresse]
8         elif op == "SUB":
9             _, reg_dest, reg_src = instruction
10            registres[reg_dest] = registres[reg_dest] - registres[reg_src]
11        elif op == "STORE":
12            _, reg, adresse = instruction
13            memoire[adresse] = registres[reg]
14        elif op == "HALT":
15            break
16    return memoire, registres

```

3. Après LOAD R0,0 : R0=20. Après LOAD R1,1 : R1=8. Après SUB R0,R1 : R0=12. Après STORE R0,2 : memoire=[20, 8, 12].

Corrigé

1NSI-ARCHOS-EVAL-B-EX2

1. Par exemple : gestion de la mémoire (allouer de l'espace à chaque programme) et gestion des fichiers (organiser le stockage sur disque).

2. Le propriétaire **peut** écrire (rw-). Le groupe **peut** écrire (rw-). Les autres **ne peuvent pas** écrire (r--).

3. rw-=6, rw-=6, r--=4 : notation octale 664.

Évaluation B — Architecture et systèmes d'exploitation

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (von Neumann, instructions); Ex. 2 : 10 pts (OS, permissions)

Exercice 1 — Von Neumann et instructions machine

(10 points)

1. (3 pts) Nommer les quatre constituants principaux de l'architecture de von Neumann, et préciser lequel exécute les instructions.
2. (4 pts) On ajoute au simulateur executer du cours l'instruction ("SUB", reg_dest, reg_src) (soustraction : $\text{reg_dest} \leftarrow \text{reg_dest} - \text{reg_src}$). Écrire le code de cette instruction.
3. (3 pts) Dérouler l'exécution de ce programme et donner l'état final de memoire :

```

1 memoire = [20, 8, 0]
2 programme = [
3     ("LOAD", "R0", 0),
4     ("LOAD", "R1", 1),
5     ("SUB", "R0", "R1"),
6     ("STORE", "R0", 2),
7     ("HALT",),
8 ]

```

Exercice 2 — Système d'exploitation et permissions

(10 points)

1. (3 pts) Citer deux fonctions assurées par un système d'exploitation.
2. (4 pts) Un fichier a les permissions "rw-rw-r--". En utilisant droit_accorde du cours, déterminer si le propriétaire peut y écrire, si le groupe peut y écrire, et si les autres peuvent y écrire.
3. (3 pts) Convertir ces permissions en notation octale avec permissions_vers_octal.

9 Réseaux : transmission de données

Corrigé

1NSI-RES-EVAL-A-EX1

- 3 paquets sont créés : "DONN" (numéro 0), "EESN" (numéro 1), "SI" (numéro 2).
- Tentative 1 : envoi de "P" avec bit 0 — échoue. Tentative 2 : retransmission de "P" avec bit 0 — réussit. Tentative 3 : envoi de "Q" avec bit 1 — réussit. Messages reçus : [(0, "P"), (1, "Q")].
- L'émetteur **retransmet** systématiquement tant qu'il ne reçoit pas l'accusé de réception attendu : un message n'est jamais considéré comme envoyé tant que sa réception n'a pas été confirmée, ce qui garantit qu'il finit par arriver (en l'absence de perte permanente).

Corrigé

1NSI-RES-EVAL-A-EX2

- peuvent_communiquer(reseau, "A", "C") renvoie True : il existe un chemin $A \rightarrow B \rightarrow C$.
-

```
1 def decider_arrosage(humidite, seuil=25):  
2     return humidite < seuil
```

- decider_arrosage(20) vaut True (sol sec, il faut arroser) ; decider_arrosage(30) vaut False (sol suffisamment humide). Le **capteur** est le capteur d'humidité du sol ; l'**actionneur** est la pompe d'arrosage.

Évaluation A — Réseaux

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (paquets, bit alterné) ; Ex. 2 : 10 pts (réseau, capteurs)

Exercice 1 — Paquets et bit alterné

(10 points)

- (4 pts) Découper le message suivant en paquets de taille maximale 4 avec decouper_en_paquets du cours :

"DONNEESNSI"

Combien de paquets sont créés, et que contiennent-ils ?

- (3 pts) On envoie les messages ["P", "Q"] avec le protocole du bit alterné ; la première tentative globale échoue. Dérouler l'envoi et donner la liste finale des messages reçus, avec leur bit.
- (3 pts) Pourquoi le protocole du bit alterné garantit-il que tous les messages finissent par arriver, même en cas de pertes ponctuelles ?

Exercice 2 — Réseau et capteurs

(10 points)

- (4 pts) On donne le réseau suivant :

{"A": ["B"], "B": ["A", "C"], "C": ["B"]}

En utilisant peuvent_communiquer du cours, vérifier que "A" et "C" peuvent communiquer.

- (3 pts) Écrire une fonction decider_arrosage(humidite, seuil=25) qui renvoie True si l'humidité est inférieure au seuil (la pompe doit s'activer).

- (3 pts) Tester cette fonction pour une humidité de 20 % et de 30 %. Identifier, dans ce système, le capteur et l'actionneur.

1NSI-RES-EVAL-B-EX1

Corrigé

- 3 paquets sont créés : "PAQUE" (numéro 0), "TSRES" (numéro 1), "EAU" (numéro 2).
- Tentative 1 : envoi de "M" avec bit 0 — réussit. Tentative 2 : envoi de "N" avec bit 1 — échoue. Tentative 3 : retransmission de "N" avec bit 1 — réussit. Tentative 4 : envoi de "O" avec bit 0 — réussit. Messages reçus : [(0, "M"), (1, "N"), (0, "O")].
- Les paquets sont envoyés **un par un** : l'émetteur attend toujours l'ACK avant d'envoyer le suivant. Un seul bit suffit donc à distinguer « un nouveau paquet » (bit différent du précédent) d'« une retransmission » (même bit que le paquet précédent, non encore confirmé).

1NSI-RES-EVAL-B-EX2

Corrigé

- peuvent_communiquer(reseau, "X", "Z") renvoie True : il existe un chemin $X \rightarrow Y \rightarrow Z$.
-

```
1 def decider_ventilation(co2, seuil=800):
2     return co2 > seuil
```

- decider_ventilation(1000) vaut True (air vicié, il faut ventiler) ; decider_ventilation(500) vaut False (air correct). Le **capteur** est le capteur de CO2 ; l'**actionneur** est le système de ventilation.

Évaluation B — Réseaux

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (paquets, bit alterné) ; Ex. 2 : 10 pts (réseau, capteurs)

Exercice 1 — Paquets et bit alterné

(10 points)

- (4 pts) Découper le message suivant en paquets de taille maximale 5 avec decouper_en_paquets du cours :

"PAQUETSRESEAU"

Combien de paquets sont créés, et que contiennent-ils ?

- (3 pts) On envoie les messages ["M", "N", "O"] avec le protocole du bit alterné ; la deuxième tentative globale échoue. Dérouler l'envoi et donner la liste finale des messages reçus, avec leur bit.
- (3 pts) Pourquoi un seul bit (0 ou 1) suffit-il à distinguer un nouveau paquet d'une retransmission, dans ce protocole ?

Exercice 2 — Réseau et capteurs

(10 points)

- (4 pts) On donne le réseau suivant :

{ "X": ["Y"], "Y": ["X", "Z"], "Z": ["Y"] }

En utilisant peuvent_communiquer du cours, vérifier que "X" et "Z" peuvent communiquer.

- (3 pts) Écrire une fonction decider_ventilation(co2, seuil=800) qui renvoie True si le taux de CO2 (en ppm) dépasse le seuil (la ventilation doit s'activer).
- (3 pts) Tester cette fonction pour un taux de 1000 ppm et de 500 ppm. Identifier, dans ce système, le capteur et l'actionneur.

10 Projet et méthodes

Corrigé

1NSI-PH-EVAL-A-EX1

1. Entre 1950 et 1990 : le premier microprocesseur commercial (1971) et l'invention du World Wide Web (1989).

2. $\frac{2}{4} \times 100 = 50,0\%$.

3. La démarche de projet est une compétence essentielle du programme (analyser un besoin, concevoir une solution, travailler en équipe, gérer le temps) qui ne peut s'acquérir qu'en la **pratiquant** réellement, pas seulement en l'étudiant en théorie : imposer un volume horaire minimal garantit que tous les élèves, quel que soit l'établissement, bénéficient effectivement de cette pratique.

Corrigé

1NSI-PH-EVAL-A-EX2

1. Le programme lève une ZeroDivisionError, avec le message division by zero.

2.

```
1 def diviser(a, b):  
2     assert b != 0, "le diviseur ne doit pas etre nul"  
3     return a / b
```

3. Une précondition explicite documente clairement, dans le code lui-même, l'hypothèse nécessaire au bon fonctionnement de la fonction, avec un **message personnalisé** compréhensible. Laisser l'erreur native se produire fonctionne aussi (le programme s'arrête bien), mais sans cette documentation, un autre développeur (ou soi-même plus tard) doit deviner pourquoi la division a été appelée avec un diviseur nul.

Évaluation A — Projet et méthodes

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (histoire, démarche de projet) ; Ex. 2 : 10 pts (débugage)

Exercice 1 — Histoire et gestion de projet

(10 points)

1. (4 pts) En utilisant evenements_entre du cours sur la chronologie [(1948, ...), (1971, ...), (1989, ...)], lister les événements situés entre 1950 et 1990.
2. (3 pts) Un projet est découpé en 4 jalons, dont 2 sont terminés. Calculer son avancement avec avancement du cours.
3. (3 pts) Pourquoi le programme officiel impose-t-il de réserver au moins un quart de l'horaire annuel à la démarche de projet, plutôt que de laisser cette part au libre choix du professeur ?

Exercice 2 — Débugage

(10 points)

```
1 def diviser_bugue(a, b):  
2     return a / b  
3  
4 diviser_bugue(10, 0)
```

1. (3 pts) Quel type d'erreur ce programme provoque-t-il ? Quel est le message exact ?
2. (4 pts) Écrire une version corrigée `diviser(a, b)` qui utilise une précondition (`assert`) pour interdire explicitement une division par zéro, avec un message clair.
3. (3 pts) Pourquoi une précondition explicite est-elle préférable à laisser l'erreur native (`ZeroDivisionError`) se produire sans explication ?

1NSI-PM-EVAL-B-EX1

Corrigé

1. Entre 1960 et 1991 : les débuts du réseau Internet (1969) et la première version publique de Python (1991).

2. $\frac{3}{5} \times 100 = 60,0\%$.

3. Avec seulement 2 à 4 élèves et un temps limité, un projet trop ambitieux risque de ne **jamais aboutir** : l'équipe s'épuise sur des fonctionnalités secondaires sans jamais disposer d'une version complète et testée. Un projet modeste mais terminé et fonctionnel est pédagogiquement plus utile (et plus valorisant) qu'un projet ambitieux resté à l'état d'ébauche.

1NSI-PM-EVAL-B-EX2

Corrigé

1. Le programme lève une `IndexError`, avec le message `list index out of range`.

2.

```
1 def acceder_element(liste, indice):
2     assert 0 ≤ indice < len(liste), "indice hors des bornes de la liste"
3     return liste[indice]
```

3. Une précondition explicite documente, directement dans le code, l'hypothèse nécessaire (un indice valide) avec un message personnalisé et compréhensible. L'erreur native `IndexError` arrête aussi le programme, mais sans expliquer la **cause métier** du problème (ici, un indice incohérent avec la taille de la liste) aussi clairement qu'un message d'assertion rédigé pour ce cas précis.

Évaluation B — Projet et méthodes

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (histoire, démarche de projet) ; Ex. 2 : 10 pts (débogage)

Exercice 1 — Histoire et gestion de projet

(10 points)

1. (4 pts) En utilisant `evenements_entre` du cours sur la chronologie [(1936, ...), (1969, ...), (1991, ...)], lister les événements situés entre 1960 et 1991.
2. (3 pts) Un projet est découpé en 5 jalons, dont 3 sont terminés. Calculer son avancement avec `avancement` du cours.
3. (3 pts) Le programme officiel précise que les professeurs veillent à ce que les projets restent « d'une ambition raisonnable ». Pourquoi cette recommandation est-elle importante pour une équipe de 2 à 4 élèves ?

Exercice 2 — Débogage

(10 points)

```
1 def acceder_element_bugue(liste, indice):
2     return liste[indice]
3
4 acceder_element_bugue([1, 2, 3], 5)
```

1. (3 pts) Quel type d'erreur ce programme provoque-t-il ? Quel est le message exact ?
2. (4 pts) Écrire une version corrigée `accéder_element(liste, indice)` qui utilise une précondition (`assert`) pour vérifier que l'indice est valide, avec un message clair.
3. (3 pts) Pourquoi une précondition explicite est-elle préférable à laisser l'erreur native (`IndexError`) se produire sans explication ?

